TxGNN vs. HGT/HAN on PrimeKG: A Reproducibility-Constrained Comparative Study

Muhammad Junaid
MS Data Science
Department of Computer Science
Email: mj2473062@gmail.com

***Abstract*—Drug repurposing models are usually judged on aggregate accuracy, but the harder and more clinically relevant test is zero-shot prediction for diseases with no approved therapy at all. TxGNN was built for exactly that case, pretraining a relational graph encoder across a biomedical knowledge graph before fine-tuning a metric-learning decoder on drug-disease links. This study set out to reproduce TxGNN against two attention-based heterogeneous GNN baselines, HGT and HAN, on PrimeKG. The original TxGNN software could not be installed in the working environment (its dependency on an old DGL release breaks under current Python), so its architecture was reimplemented from the published description in PyTorch Geometric instead of loading the authors' checkpoint. All three models were trained on identical random and zero-shot drug-disease splits built for this study. The result was not the clean separation reported in the original paper: TxGNN's reimplementation, HGT, and HAN all scored between 0.977 and 0.996 AUPRC/AUROC on both splits, a band too narrow to support a confident ranking. Reduced model scale (forced by a single T4 GPU's memory limit on the HGT/HAN runs) and an evaluation protocol using uniform random negative sampling are the two most likely explanations, and both are discussed as limitations rather than treated as settled.**

***Index Terms*—Drug Repurposing, Knowledge Graphs, Graph Neural Networks, TxGNN, Heterogeneous Graph Transformer, HGT, HAN, PrimeKG, PyTorch Geometric, Link Prediction**

I. Introduction

Most of the difficulty in drug repurposing isn't predicting a treatment for a well-studied disease— it's predicting one for a disease that has never had an approved therapy at all. Of PrimeKG's 17,080 disease nodes, a large majority fall into that category. A model that performs well only on diseases it has already seen labeled drugs for is not actually solving the repurposing problem; it is solving a much easier one. TxGNN (Huang et al., 2024) was built around this distinction directly, using a two-phase training procedure—self-supervised pretraining across the full knowledge graph, then fine-tuning a decoder for the indication/contraindication task—specifically to handle diseases absent from the fine-tuning labels.

This study asks whether that two-phase design actually outperforms a more conventional single-stage, attention-based heterogeneous GNN (HGT, HAN) when run on the same data and the same splits. It is a narrower, more tractable stand-in for the broader question of whether knowledge-graph-aware, metric-learning architectures beat generic attention mechanisms—the same self-attention mechanism that underlies transformer language models, just applied here to a graph instead of a token sequence.

II. Background and Related Work

A. PrimeKG

PrimeKG (Chandak, Huang & Zitnik, 2023) integrates more than 20 biomedical resources into one heterogeneous graph spanning diseases, drugs, genes/proteins, phenotypes, anatomy, pathways, biological processes, and exposures. The instance downloaded for this study reports 17,080 disease nodes and 7,957 drug nodes, matching the published statistics, across 129,375 total nodes and 8,100,498 total edges (see the data card in the Appendix).

B. TxGNN

TxGNN (Huang, Chandak, Wang, Havaldar, Vaid, Leskovec, Nadkarni, Glicksberg, Gehlenborg & Zitnik, 2024, *Nature Medicine*) frames drug repurposing as link prediction across two relation types, indication and contraindication, and trains in two phases: self-supervised pretraining over the entire knowledge graph, then fine-tuning a metric-learning decoder on the labeled task. The paper reports a large improvement over prior methods under zero-shot evaluation, though the exact magnitude could not be pinned down with confidence in this session—two different automated readings of the full text returned different numbers (49.2% / 35.1% AUPRC gain for indications/contraindications in one reading, 19.0% / 23.9% in another), and that discrepancy is left open here rather than resolved by picking whichever number sounds better.

C. HGT

Heterogeneous Graph Transformer (Hu, Dong, Wang & Sun, 2020, WWW '20) generalizes transformer-style multi-head attention to heterogeneous graphs, learning separate attention parameters per node-type/edge-type combination. It has no built-in pretrain phase; the encoder is trained directly on whatever supervised task it is pointed at.

D. HAN

Heterogeneous Graph Attention Network (Wang, Ji, Shi, Wang, Cui, Yu & Ye, 2019, WWW '19) takes a metapath-based approach, computing attention over node-level neigh-

bors within a metapath and then over the metapaths themselves. Like HGT, it is single-stage and was used as a baseline in TxGNN's original evaluation.

E. HGTDR

HGTDR (*Bioinformatics*, 2024, DOI 10.1093/bioinformatics/btae349) applies HGT specifically to drug repurposing on PrimeKG, augmenting node features with BioBERT/ChemBERTa embeddings. It is the closest published comparable to this study's HGT baseline; this study does not reuse its code, building HGT independently via PyTorch Geometric's `HGTConv` instead.

F. PyTorch Geometric

PyG (Fey & Lenssen, 2019) is the library underlying every model trained in this study—the TxGNN reimplementation, HGT, and HAN all share its `HeteroData` graph representation and message-passing primitives, which is itself part of why a fair architectural comparison was feasible at all without three separate codebases.

III. Case Study from the Original Paper

The original TxGNN paper includes case studies where the model recommended drugs that were not obvious from disease taxonomy alone. One documented case (verified against a PubMed Central mirror of the published article, PMC11326339, retrieved 2026-06-29) involves Kleefstra syndrome, a neurodevelopmental disorder with speech delay and autism-spectrum features:

> "On querying the TXGNN Predictor, zolpidem was recommended as the number one drug repurposing candidate."

Zolpidem is ordinarily a sedative—recommending it for a neurodevelopmental disorder looks wrong on its face. The paper's explainer module reportedly surfaced multi-hop graph paths suggesting a paradoxical stimulative effect on prefrontal cortex function in some neurological conditions, which is offered as the mechanistic rationale rather than the model arbitrarily picking a sedative.

This case study matters for this study's results in a specific way: it is exactly the kind of counterintuitive, mechanism-dependent prediction that a metric-learning decoder reading multi-hop graph structure is supposed to be good at, and that a baseline trained only to fit observed indication/ contraindication edges is not obviously built to surface. None of the three models trained here were evaluated on this specific case (Kleefstra syndrome was not isolated as a held-out test case in either split), so this study cannot confirm or contradict that this study's TxGNN reimplementation reproduces that specific prediction. That is a limitation, not a finding, and is listed again in Section VII.

IV. Methods

A. Environment

Training ran on a remote Google Colab GPU runtime (NVIDIA T4, approximately 14.5–14.6 GB peak memory observed), accessed via VS Code's Colab extension; the local machine (Windows, approximately 4 GB RAM, frequently near disk-full) ran only the lightweight aggregation script. Python on Colab is 3.12; the original TxGNN pip package depends on `distutils`, removed from the standard library in Python 3.12, and fails to install (`setup.py egg_info` error). DGL and the TxGNN package were dropped entirely; PyTorch Geometric covers both the TxGNN reimplementation and the HGT/HAN baselines.

B. Data

PrimeKG was downloaded directly from Harvard Dataverse (DOI 10.7910/DVN/IXA7BM) via its file API rather than a hardcoded file ID. Indication and contraindication relation names were located by substring match on the live data (`'indicat'` / `'contraindicat'`) rather than assumed in advance.

C. TxGNN Reimplementation

A relational encoder (2-layer `HeteroConv` with `SAGEConv` per edge type, hidden dimension 64, learnable per-node-type embeddings as input features since PrimeKG ships none) was pretrained for 50 epochs via link-prediction loss sampled across all non-target relation types, then fine-tuned for 200 epochs on indication and contraindication training pairs with a dot-product decoder.

This is a documented simplification of the published architecture: the decoder is dot-product rather than TxGNN's published metric-learning module, and the encoder uses mean aggregation (`SAGEConv`) rather than the original's specific message-passing design. The dot-product decoder was deliberately kept identical to the HGT/HAN baselines' decoder so the comparison isolates the encoder's training paradigm rather than decoder differences.

D. HGT/HAN Baselines

Both use PyTorch Geometric's `HGTConv` and `HANConv` respectively, single-stage supervised training directly on the same indication/contraindication train pairs, with the same dot-product decoder.

The first run out-of-memory'd a T4 GPU doing full-batch attention over PrimeKG's 8.1 million edges.

The fix included:

- Hidden dimension reduced from 64 to 32.
- Attention heads reduced from 4 to 2.
- Each relation type capped at 50,000 randomly sampled edges (seed 42).
- Training reduced from 200 epochs to 100 epochs.
- Mixed-precision (FP16) forward passes enabled.

This means HGT/HAN ran at a smaller scale than the TxGNN reimplementation—acknowledged directly as a limitation in Section VII, not hidden in a footnote.

E. Data Splits

Built for this study, not the original TxGNN splits (the original used `TxData`, part of the dropped package).

Random Split:

- 70/15/15 train/validation/test.
- Applied independently over indication and contraindication pairs.
- Random seed = 42.

Zero-Shot Split:

- 20% of diseases with at least one indication/contraindication edge selected at random.
- All edges belonging to those diseases moved entirely to the test set.
- Remaining edges split 85/15 into train and validation.
- Random seed = 42.

Both TxGNN and HGT/HAN were evaluated against identical split arrays (saved as `splits.npz` after the TxGNN run and loaded by the HGT/HAN notebook with an assertion that node counts match).

F. Evaluation Metrics

Performance was evaluated using:

- Area Under the Precision–Recall Curve (AUPRC), computed using `sklearn.metrics.average_precision_score`.
- Area Under the Receiver Operating Characteristic Curve (AUROC), computed using `sklearn.metrics.roc_auc_score`.

Metrics were calculated using the positive drug–disease test pairs together with an equal number of uniformly sampled negative (non-edge) drug–disease pairs.

V. Results

All metrics below are this study's own runs—see the Appendix for the complete raw `results.json`. None are paper-reported numbers; where the original paper's numbers are discussed, they are explicitly labeled as such in Sections II and VI.

A. Random Split

B. Zero-Shot Split

The TxGNN reimplementation has the best indication AUPRC on the random split (0.9911 vs. 0.9834 and 0.9818) but not on the zero-shot split, where HAN edges it out on indication AUPRC (0.9853 vs. 0.9799) and HGT leads on contraindication AUPRC (0.9935 vs. 0.9832).

Across both splits, no model is consistently best on every metric, and every score sits inside a 0.977–0.996 band. That is a narrower spread than this study expected going in.

VI. Discussion

A. Do attention-based heterogeneous GNNs perform better with a knowledge graph?

Not demonstrably, based on these runs. The three models land too close together to support a ranking on either split.

That contradicts the separation the original TxGNN paper reports against HGT/HAN baselines under zero-shot evaluation—though, as noted in Section II, this study could not pin down the original paper's exact improvement percentages with confidence, so "contradicts" should be read as "does not reproduce the qualitative pattern," not as a precise numerical comparison.

B. Is two-phase training the better approach?

The architectural difference was real in this study's setup—the TxGNN reimplementation pretrained across the full graph before fine-tuning, whereas HGT/HAN did not.

However, the results do not demonstrate an advantage from the two-phase training strategy.

The most likely explanation lies in the evaluation protocol. Uniform random negative sampling makes most negative samples easy to separate from positives regardless of how well the encoder generalizes, compressing all three models toward nearly identical performance.

A harder negative-sampling strategy, such as degree-matched or type-matched negatives, would likely provide a more informative comparison than additional hyperparameter tuning.

C. Why does zero-shot matter?

Independent of this study's inconclusive comparison, the motivating fact remains unchanged: most of PrimeKG's 17,080 diseases have no recorded therapy.

A repurposing model that only performs well for diseases with existing labeled drugs addresses a substantially easier and less clinically useful problem than the one drug repurposing is intended to solve.

D. Why is attention optional in TxGNN but central in HGT/HAN?

This study's reimplementation used `SAGEConv` (mean aggregation without attention) for the TxGNN encoder.

Consequently, the comparison conducted here is more accurately described as

> "two-phase pretraining and fine-tuning versus single-stage supervised training"

rather than

> "attention versus non-attention."

Answering the latter question directly would require implementing an attention-based encoder inside the TxGNN architecture, which was outside the scope of this work.

TABLE I
PERFORMANCE ON THE RANDOM SPLIT

Model	Indication AUPRC	Indication AUROC	Contraindication AUPRC	Contraindication AUROC	Parameters	Training Ti
TxGNN reimplementation (this study)	0.9911	0.9934	0.9933	0.9956	9,105,600	1.1
HGT (this study, reduced scale)	0.9834	0.9859	0.9945	0.9958	4,327,100	1.5
HAN (this study, reduced scale)	0.9818	0.9861	0.9812	0.9866	4,169,696	0.6

TABLE II
PERFORMANCE ON THE ZERO-SHOT SPLIT

Model	Indication AUPRC	Indication AUROC	Contraindication AUPRC	Contraindication AUROC
TxGNN	0.9799	0.9859	0.9832	0.9892
HGT	0.9769	0.9830	0.9935	0.9947
HAN	0.9853	0.9883	0.9855	0.9903

VII. LIMITATIONS

The present study has several important limitations.

- **Not the original TxGNN.** The official TxGNN package could not be installed under the current Python environment. Consequently, every result reported as "TxGNN" is based on a PyTorch Geometric reimplementation employing a simpler dot-product decoder and a SAGEConv encoder.
- **Unequal model scale.** HGT and HAN were executed using a reduced hidden dimension (32) together with 50,000 sampled edges per relation to avoid GPU memory exhaustion. The TxGNN reimplementation retained hidden dimension 64 and the complete PrimeKG graph, introducing an unavoidable fairness limitation.
- **Random negative sampling.** Training and evaluation both relied on uniformly sampled non-edge pairs. Such negatives are widely recognized as being substantially easier than hard negatives and therefore tend to inflate AUPRC and AUROC values.
- **Independent train/test splits.** The experimental splits were constructed specifically for this study and are not identical to the official TxGNN benchmark.
- **No disease-area holdout experiment.** The optional disease-area evaluation planned for the project was not completed.
- **Case study not reproduced.** The Kleefstra syndrome example presented in Section III was not evaluated experimentally.
- **Citation discrepancy.** The exact zero-shot improvement percentages reported in the original paper could not be reconciled because two independent automated readings produced different values.

VIII. CONCLUSION AND FUTURE WORK

This study compared a reimplemented version of TxGNN against HGT and HAN using identical PrimeKG train/test splits.

The experimental results did not demonstrate a clear performance advantage for any architecture.

All three models produced remarkably similar AUPRC and AUROC values under both random and zero-shot evaluation. The most plausible explanation is that uniformly sampled negative examples are insufficiently challenging to distinguish between heterogeneous graph learning architectures, combined with unavoidable scaling differences introduced by GPU memory limitations.

Future work should therefore focus on

- hard negative sampling,
- degree-aware evaluation,
- full-scale HGT/HAN experiments on larger-memory GPUs,
- reproducing the official TxGNN benchmark splits,
- implementing the original metric-learning decoder,
- evaluating additional biomedical knowledge graphs beyond PrimeKG.

ACKNOWLEDGMENT

The author acknowledges the use of Google Colab for GPU-based experimentation, PyTorch Geometric for heterogeneous graph implementation, and PrimeKG for providing the biomedical knowledge graph used throughout this study.

REFERENCES

[1] P. Chandak, K. Huang, and M. Zitnik, "Building a knowledge graph to enable precision medicine," *Scientific Data*, vol. 10, 2023. Dataset DOI: 10.7910/DVN/IXA7BM.

[2] K. Huang, P. Chandak, Q. Wang, S. Havaldar, A. Vaid, J. Leskovec, G. N. Nadkarni, B. S. Glicksberg, N. Gehlenborg, and M. Zitnik, "A foundation model for clinician-centered drug repurposing," *Nature Medicine*, 2024. DOI: 10.1038/s41591-024-03233-x.

[3] Z. Hu, Y. Dong, K. Wang, and Y. Sun, "Heterogeneous Graph Transformer," in *Proceedings of The Web Conference (WWW)*, 2020, pp. 2704–2710.

[4] X. Wang, H. Ji, C. Shi, B. Wang, P. Cui, P. Yu, and Y. Ye, "Heterogeneous Graph Attention Network," in *Proceedings of The World Wide Web Conference*, 2019, pp. 2022–2032.

[5] "Advancing Drug Repurposing with Heterogeneous Graph Transformers," *Bioinformatics*, vol. 40, no. 7, 2024, Article btae349.

[6] M. Fey and J. E. Lenssen, "Fast Graph Representation Learning with PyTorch Geometric," ICLR Workshop on Representation Learning on Graphs and Manifolds, 2019.

APPENDIX A
RAW EXPERIMENTAL RESULTS

The complete experimental outputs were stored in a JSON file (`results.json`), which served as the source for all tables reported in Section V.

Because the JSON file is lengthy, it is recommended to include it using the `listings` package.

Add the following package in the preamble if not already included:

```
\usepackage{listings}
```

Then include the appendix as

Listing 1. Raw Experimental Results

```
{
  ...
  (Paste the complete JSON exactly as
  provided in your Markdown file.)
  ...
}
```

APPENDIX B
DATA CARD

PrimeKG statistics used in this study are summarized below.

TABLE III
PRIMEKG DATASET SUMMARY

Property	Value
Disease Nodes	17,080
Drug Nodes	7,957
Total Nodes	129,375
Total Edges	8,100,498

APPENDIX C
REPRODUCIBILITY NOTES

The implementation used throughout this work differs from the original TxGNN implementation in several respects.

- PyTorch Geometric was used instead of DGL.
- The official TxGNN package could not be installed because of Python 3.12 dependency issues.
- HGT and HAN were implemented using native `HGTConv` and `HANConv` layers.
- Experiments were executed on Google Colab using an NVIDIA T4 GPU.
- Identical train/test splits were reused for all models.